

Antenna AI Designer 项目指南

1. 一句话理解这个项目

这是一个“让 AI 大模型在天线设计考试中做题、由 HFSS 仿真软件自动判卷”的项目：

- 1. 我们先用一个 JSON 文件把题目写得清清楚楚（考什么、哪些量能改、哪些量不许改、怎么给分）；
- 2. 让大模型（LLM）像考生一样，提交一整套天线尺寸参数；
- 3. 项目里的可信 Python 代码拿着这些参数，自动在 Ansys HFSS 里建模、仿真、读结果；
- 4. 一个独立的评测器根据题目 JSON 给这个设计打分；
- 5. 模型看到分数对应的实测结果后继续改进参数，直到预算用完或主动交卷；
- 6. 所有过程和结果都被完整地存到硬盘上，形成可追溯的历史记录。

2. 基础背景

在读代码和结果之前，只需要先知道以下几个概念。

名词	通俗解释
AEDT	Ansys Electronics Desktop，电磁仿真软件的“总桌面程序”。
HFSS	AEDT 里的高频电磁场求解器，本项目真正用来算天线的部分。可以理解为“AEDT 里的计算引擎”。
PyAEDT	Python 库，让 Python 程序能自动操作 AEDT（建模、求解、读曲线）。
贴片天线	一块金属“补丁”贴在介质板上，是最常见的平面天线。
S11 / 回波损耗	天线的输入信号中反射回去的功率占比。dB 值越负越好，例如 -20 dB 比 -10 dB 好。
增益 dBi	天线朝最强方向辐射能量的能力，越大越好。
轴比 AR	圆极化天线的“圆不圆”，越接近 0 dB 越理想，一般 ≤3 dB 算可用圆极化。
候选 candidate	模型提交的一套完整设计参数，以及它对应的完整仿真结果。 一次设计可能产生多个候选。

不需要会手动操作 AEDT 界面也能使用本项目：所有建模和求解都由代码自动完成。AEDT 窗口只是偶尔用来人工检查。

3. 一次完整设计是怎么跑起来的



几个关键设计原则：

1. **模型不能写代码。** 它只能调用两个工具：`create*_antenna(参数)` 提交一个候选，`finalize_design` 交卷。几何、端口、边界、求解设置全部由项目内固定代码生成。
2. **自然语言不是权威。** 题目里的 `natural_language_description` 只是给人看的补充说明，真正算数的是结构化字段（阈值、单位、仿真控制）。
3. **裁判独立。** 评测器不问模型“你觉得自己考得怎么样”，只看 HFSS 实测值。
4. **失败不覆盖成功。** 第 3 个候选失败不会抹掉第 1、2 个候选的成绩；最终选“所有完整成功候选里分数最高的那个”。

4. 仓库文件地图

4.1 根目录代码

文件/目录	作用	你什么时候会碰它
main.py	程序入口，解析命令行参数，组织单模型/多模型、单题/多题流程	想换运行方式时
config.py	全局配置（API 地址、模型名、迭代上限、AEDT 路径等）	换模型或环境时
task_spec.py	题目公共合同的数据结构与校验	改题目结构时
task_batch.py	多题目清单 task_batch.json 的加载与校验	加/减题目时
model_batch.py	多模型清单 model_batch.json 的加载与校验	加/减参赛模型时
design_loop.py	候选生成主循环：调模型 -> 调后端 -> 记录	改迭代逻辑时
design_spec.py	候选参数的严格模型与 JSON Schema	改可调参数字段时
prompts.py	发给模型的系统提示词、工具 schema	改模型行为协议时
model_client.py	对接各类 LLM API（Responses / Chat Completions / Anthropic）	接新模型时
hfss_client.py	真正驱动 PyAEDT 建模、求解、读数的“可信后端” （本项目最大的文件）	改建模/仿真/ 读数时
evaluator.py	独立评分器，选出最佳候选并生成评测报告	改评分规则时
verify_min.py	最小连通性测试：确认 Python 能连上本机 AEDT	环境出问题时

4.2 配置与题目

文件/目录	作用
tasks/	题目合同 ，每道题一个 JSON，是本项目最核心的“考卷”
task_batch.json	题目清单：按顺序声明启用哪些题
model_batch.json	模型清单：按顺序声明启用哪些模型
job_*.yaml	各模型的连接配置（API key、模型名等，注意不要提交真实密钥）
.env.example	环境变量示例（复制为 .env 后填写）

文件/目录	作用
requirements.txt	Python 依赖清单

4.3 结果与示例

目录	作用
hfss_projects/	所有历史运行结果，详见第 7 章
full_score_example/	每道题的“满分见证工程”，证明题目确实存在能拿满分的解
logs/	早期运行的零散日志（现在主流程已改为按 run/batch 目录落盘）
tests/	离线单元测试，不需要启动 AEDT
frameworks/	按天线拓扑拆分的设计框架：参数 schema、几何约束、参数解析

5. 题目部分

5.1 题目文件在哪里

所有题目都在 tasks/ 目录下，目前有三道：

文件	task_id	通俗描述
tasks/inset_patch_2p45.json	inset_patch_2p45_v6	2.45 GHz 内嵌馈电矩形贴片
tasks/dual_band_patch_1p9_2p45_v1.json	dual_band_patch_1p9_2p45_v1	1.9/2.45 GHz 双频贴片
tasks/cp_patch_gps_1p575_v1.json	cp_patch_gps_1p575_v1	GPS L1 频段右旋圆极化贴片

5.2 题目 JSON 的整体骨架

每个题目文件都是同一个固定骨架，从上到下依次是：

```
{
  "schema_version":      合同格式版本
  "task_id":             题目唯一 ID（结果目录、报告文件名都用它）
  "title":               题目标题（给人看的一句话概括）
  "natural_language_description": 自然语言题目描述（只是补充，不参与判分）

  "topology":            天线拓扑：用什么 builder、几个端口
  "calibration":         满分校准状态：这道题有没有被验证过存在满分解
  "fixed_parameters":    固定量：材料、基板尺寸等模型不许改的参数
  "design_variables":     可调变量：模型真正要“设计”的尺寸及其取值范围
  "simulation_control":   仿真控制：扫频、求解设置、空气盒、远场球等
  "objectives":          评分目标：每个指标怎么量、怎么算分
}
```

5.3 逐字段解读（以 2.45 GHz 题为例）

schema_version

合同格式的版本号。目前为 1。代码据此判断自己能不能读懂这份合同。

task_id

题目的唯一身份证号，例如 `inset_patch_2p45_v6`。它的作用：

- 结果子目录用它命名，例如 `01_inset_patch_2p45_v6/`；
- 批次汇总报告用它命名，例如 `batch_results_inset_patch_2p45_v6.json`；
- 评测报告里用它回溯“这是哪道题的成绩”。

注意：`task_batch.json` 里每一项的 `id` 是清单里的短 ID（如 `inset_patch_2p45`），而题目 JSON 内部的 `task_id` 带版本后缀（如 `inset_patch_2p45_v6`），两者不必完全相同，真正落盘用的是题目 JSON 里的 `task_id`。

title

题目标题，一句话说明“考什么”，例如“2.45 GHz 受限尺寸内嵌馈电矩形贴片天线”。它主要用于人快速识别，不参与判分。

natural_language_description

自然语言版本的题目要求，写给人和模型阅读。它必须与结构化字段一致，但**不是权威**：

- 如果自然语言说“带宽不小于 120 MHz”，而 `objectives` 里写的是 110.0，以 110.0 为准；自然语言不会被反向解析成阈值。

这个设计是为了避免模型钻空子：所有数字、单位、语义都锁在结构化字段里，评测器只认结构化字段。

topology

声明天线“长什么样”：

```
"topology": {
  "id": "inset_fed_rectangular_patch",
  "port_count": 1,
  "description": "单层矩形微带贴片，居中内嵌微带馈线，一个 50 欧姆集总端口"
}
```

- id 是关键：程序用它在 frameworks/ 里自动找到对应的可信 builder（建模代码）、参数 schema 和提示词约束。
- port_count 声明天线端口数，后端据此生成端口和 S 参数读取逻辑。

目前支持的拓扑 ID：

topology.id	对应框架文件	说明
inset_fed_rectangular_patch	frameworks/inset_fed_rectangular_patch.py	内嵌馈电矩形贴片
dual_band_coax_patch	frameworks/coax_fed_rectangular_patch.py	同轴馈电双频矩形贴片
single_feed_cp_coax_patch	frameworks/coax_fed_rectangular_patch.py	单馈同轴圆极化贴片

新增拓扑时，除了写新题目 JSON，还要在 frameworks/ 下新增对应框架文件和 hfss_client.py 中的 builder 注册，否则任务加载会失败。

calibration

记录这道题“有没有被证明存在满分解”：

```
"calibration": {
  "status": "verified",
  "full_score_witness_verified": true,
  "verification_date": "2026-08-25",
  "solver": "Ansys Electronics Desktop Student 2025 R2 / HFSS 2025.2",
  "notes": "...
}
```

为什么要这个字段？因为出题时如果把满分线定得太高，可能所有模型都拿不到满分，却说不清是“模型不行”还是“题目本身无解”。所以流程是：

- 1. 先冻结题目合同和 HFSS 版本/设置；
- 2. 由人工或传统优化器找到一个满分解（见证解）；
- 3. 用与正式评测完全相同的代码链路复算一遍；
- 4. 满分阈值要放在见证解性能以内并留出数值余量；
- 5. 见证工程保存在 full_score_example/<题目>/ 下，但不给模型看。

状态含义：

status	含义
verified	已在同一固定环境中验证过满分解
uncalibrated	阈值只是占位线，题目还不适合正式排行

fixed_parameters

模型只能读、不能改的量。以 2.45 GHz 题为例：

```
"fixed_parameters": {
  "substrate_material":      { "value": "Rogers5880_custom", ... },
  "substrate_relative_permittivity": { "value": 2.2, ... },
  "substrate_loss_tangent":  { "value": 0.0009, ... }
}
```

每个固定量都带 value / unit / description ，例如相对介电常数是无量纲的 1 。圆极化题里还固定了基板尺寸、探针半径、端口半径等。

作用：固定评测环境，保证所有模型在完全相同的物理条件下竞争。

design_variables

模型真正要设计的自由度，也是“考题的核心”。每个变量都声明单位、上下限和含义：

```
"design_variables": {
  "substrate_width_mm": { "unit": "mm", "minimum": 60.0, "maximum": 100.0, ... },
  "patch_width_mm":    { "unit": "mm", "minimum": 20.0, "maximum": 80.0, ... },
  "feed_width_mm":     { "unit": "mm", "minimum": 0.2,  "maximum": 20.0, ... },
  ...
}
```

规则：

- 模型每次调用工具时，必须一次性给出全部字段，不能缺省，不能多给；
- 只能传数字，不能传 "5mm" 这种字符串；

- 超出上下限、或几何上互相矛盾（例如馈线放不进基板）会被直接拒收。

simulation_control

评测端的“考场规则”，模型只能读不能改，包括：

- 自适应求解频率、扫频起止/点数/类型；
- 效率扫频（`efficiency_sweep`）窗口；
- 空气盒大小（`air_margin_xy` / `air_above` / `air_below`）；
- 端口阻抗；
- 求解收敛设置（`setup`：最大 `delta S`、最大/最小迭代次数等）；
- 远场球角度范围（`far_field`：`theta/phi` 起止和步进）；
- 是否保存场数据（`save_fields` / `save_rad_fields`）。

这些参数保证“同一个分数在不同模型之间可比”。修改它们等于换了考场，历史成绩不能直接对比。

objectives

判卷标准，每个指标一项。这是题目里最重要的部分。以 `resonant_frequency` 为例：

```
"resonant_frequency": {
  "metric_key": "center_freq_ghz",          // 对应实测指标的字段名
  "display_name": "Resonant frequency at minimum S11",
  "unit": "GHz",                             // 数值单位
  "value_representation": "linear_frequency", // 数值语义（线性频率，不是角频率）
  "goal": "target_with_tolerance",           // 目标类型：落在目标附近
  "target": { "value": 2.45, "unit": "GHz" },
  "tolerance": { "value": 0.025, "unit": "GHz" },
  "zero_score_tolerance": { "value": 0.075, "unit": "GHz" },
  "points": 25,                             // 这一项满分 25 分
  "semantics": "固定 2.30-2.60 GHz 扫频内 dB(S11) 最小值对应的频率，单位线性 GHz"
}
```

常见的 `goal` 类型：

goal	含义	典型指标
target_with_tolerance	越接近目标越好，容差内得满分	谐振频率
at_most	数值越小越好	S11、轴比
at_least	数值越大越好	增益、带宽、效率、RHCP 占优

计分方式有两种：

- 1. **单段计分**（旧格式）：零分线到合格线之间线性给分；
- 2. **三段计分**（含 `full_score_threshold / full_score_tolerance` 的目标）：零分线到合格线贡献 0–60%，合格线到满分线贡献 60–100%。

`value_representation` 字段值得特别留意，它防止 dB/线性、增益/ `realized gain` 这类常见混淆。例如 `10_log10_power_gain_relative_to_isotropic` 明确表示“dBi 的对数功率增益”。

5.4 三道题分别在考什么

题目	可调变量（举例）	核心目标（举例）
内嵌馈电 2.45 GHz 贴片	基板长宽厚、贴片长宽、馈线宽、内嵌深度、槽隙	谐振对准 2.45 GHz、S11 足够低、带宽够宽、增益和效率达标
1.9/2.45 GHz 双频贴片	贴片几何、同轴馈点位置等	两个频段都要谐振、阻抗匹配和增益达标
GPS L1 圆极化贴片	贴片长宽、馈点 x/y 偏移	1.575 GHz 谐振、轴比小、右旋圆极化占优、RHCP 增益达标

5.5 想改题目时怎么动手

常见改动及注意事项：

想改什么	改哪里	注意
放宽/ 收紧某个阈值	<code>tasks/*.json</code> 对应 <code>objectives</code> 字段	改完要同步更新 <code>calibration</code> ，必要时重新找见证解
扩大设计空间	<code>design_variables</code> 的 <code>min/max</code>	太大会让题目变难，且需确认几何约束不会自相矛盾
换材料	<code>fixed_parameters</code> 里的介电常数/ 损耗角	这会改变所有历史成绩的可比性
改扫频范围	<code>simulation_control</code>	覆盖所有指标的测量窗口，否则读数会被截断
加新题	新建 <code>tasks/xxx.json</code> 并在 <code>task_batch.json</code> 注册	还要在 <code>frameworks/</code> 有对应拓扑支持

经验法则：只改题目 JSON 不够时，才去改 `frameworks/` 或 `hfss_client.py`；能不动可信后端就不要动。

6. 批次配置： task_batch.json 与 model_batch.json

6.1 task_batch.json ： 考卷清单

```
{
  "schema_version": 1,
  "tasks": [
    { "id": "inset_patch_2p45",      "spec_file": "tasks/inset_patch_2p45.json",      "enabled": true },
    { "id": "dual_band_patch_1p9_2p45", "spec_file": "tasks/dual_band_patch_1p9_2p45_v1.json", "enabled": true },
    { "id": "cp_patch_gps_1p575",      "spec_file": "tasks/cp_patch_gps_1p575_v1.json",      "enabled": true },
  ]
}
```

- 按 tasks 数组顺序依次执行；
- enabled: false 的题会被跳过；
每道题拿到独立的候选预算（--max-iterations）。

6.2 model_batch.json ： 考生清单

```
{
  "schema_version": 1,
  "models": [
    { "id": "glm-4.5", "config_file": "job_glm_4_5.yaml", "api_style": "chat_completions", "enabled": true },
    { "id": "glm-5",   "config_file": "job_glm_5.yaml",   "api_style": "chat_completions", "enabled": true },
  ]
}
```

- id 只用于结果目录命名和汇总，不会发给模型；
- api_style 支持 responses、chat_completions、anthropic_messages；
- 批次严格串行：每个模型跑完并关闭 AEDT 后，再启动下一个，避免工程状态互相污染。

两者可以组合：--model-batch × --task-batch 会形成“每个模型 × 每道题”的完整矩阵，每个组合都有独立的候选与求解预算。

7. 历史记录存储结构（重点）

所有运行结果都在 hfss_projects/ 下。这里有两类目录：

```
hfss_projects/  
  run_20260825_115033_352903/      <- 单模型运行（一次 python main.py 的产物）  
  batch_20260828_191835_257084/    <- 多模型批次（--model-batch 的产物）  
  ...（目录名 = 类型 + UTC 时间戳，永远不覆盖旧结果）
```

目录名里的时间戳来自 UTC，所以和本地时间（例如北京时间）相差 8 小时左右。这是刻意的：保证每次运行都有独立目录，历史永远可追溯。

7.1 结构总览： run_* 单模型运行

单题运行时，候选文件和报告直接放在 run 根目录：

```
hfss_projects/run_20260825_115033_352903/  
  task_spec.json                  <- 本次使用的题目合同快照（权威副本）  
  candidate_001.aedt              <- 第 1 个候选的 HFSS 工程（可双击用 AEDT 打开）  
  candidate_001.aedtresults/      <- 第 1 个候选的求解结果数据  
  candidate_001.pyaedt/           <- PyAEDT 自动化缓存  
  candidate_001_metrics.json      <- 第 1 个候选的带单位设计参数与实测指标  
  candidate_002.aedt ...  
  design_log.json                 <- 完整对话与每轮工具返回（模型视角的“草稿纸”）  
  run_manifest.json               <- 所有候选的结构化清单（后端视角的“过程记录”）  
  evaluation_report.json          <- 最终评分报告（裁判视角的“成绩单”）  
  resource_usage.json             <- API 延迟、token 用量、求解耗时  
  manual_verification.json        <- 人工复核步骤（如何在 AEDT 里验证选中工程）
```

多题运行（--task-batch，单模型）时，先按题目分子目录，并多一个总索引：

```
hfss_projects/run_20260827_115045_126815/  
  run_results.json                <- 总索引：每道题的得分和报告路径  
  01_dual_band_patch_1p9_2p45_v1/  
    candidate_001.aedt  
    candidate_001_metrics.json  
    task_spec.json  
    design_log.json  
    run_manifest.json  
    evaluation_report.json  
    resource_usage.json  
    manual_verification.json  
  02_cp_patch_gps_1p575_v1/  
  ...
```

7.2 结构总览： batch_* 多模型批次

多模型批次的层级是 **批次** → **模型** → **题目** → **候选**：

```
hfss_projects/batch_20260828_191835_257084/
  model_batch.json          <- 本次批次使用的模型清单快照
  batch_results.json        <- 批次总索引：任务清单和每题报告路径
  batch_results_inset_patch_2p45_v6.json      <- 第 1 题：各模型成绩与排名
  batch_results_dual_band_patch_1p9_2p45_v1.json <- 第 2 题
  batch_results_cp_patch_gps_1p575_v1.json    <- 第 3 题

01_glm-4.5/                <- 模型 1（编号_id）
  01_inset_patch_2p45_v6/    <- 模型 1 的题目 1
    candidate_001.aedt
    candidate_001.aedtresults/
    candidate_001.pyaedt/
    candidate_001_metrics.json
    candidate_002.aedt ...
    task_spec.json
    design_log.json
    run_manifest.json
    evaluation_report.json
    resource_usage.json
    manual_verification.json
  02_dual_band_patch_1p9_2p45_v1/
  03_cp_patch_gps_1p575_v1/

03_glm-5/                  <- 模型 2（注意编号按清单位置，跳过禁用模型）
  01_inset_patch_2p45_v6/
  02_dual_band_patch_1p9_2p45_v1/
  03_cp_patch_gps_1p575_v1/
```

命名规则：

层级	命名	说明
批次根目录	batch_<UTC 时间戳>	一次 --model-batch 运行
模型目录	<两位序号>_<模型 id>	例如 01_glm-4.5 ； 被禁用的模型不生成目录
题目目录	<两位序号>_<task_id>	例如 03_cp_patch_gps_1p575_v1
候选工程	candidate_<三位序号>.aedt	从 001 开始，按提交顺序递增

小提示：01_glm-4.5 和 03_glm-5 中间没有 02_glm-4.7，是因为 4.7 在 model_batch.json 里 enabled: false，被跳过但保留序号占位。

7.3 每个候选的文件含义

以 candidate_009.aedt 为例，同名的一组文件/目录属于同一个候选：

文件/目录	是什么	怎么用
candidate_009.aedt	HFSS 工程文件	双击（或用 AEDT 打开） 可看到完整几何、端口、设置
candidate_009.aedtresults/	HFSS 求解结果数据库（网格、 矩阵、场）	AEDT 打开工程时自动加载； 不要手动改名
candidate_009.pyaedt/	PyAEDT 自动化产生的缓存目录	一般不用管
candidate_009_metrics.json	这个候选的“成绩条”：设计参数、 带单位的实测指标、原始读数	快速看结果优先打开它

*_metrics.json 里最有用的字段：

- design_parameters：这套候选的输入尺寸（带单位）；
- metrics：简化后的关键数值，例如 s11_min_db、peak_gain_dbi；
- measured_metrics：每个指标的完整定义（值、单位、value_representation）；
- 原始 HFSS 读数：用于追溯数值从哪条曲线、哪个频点读出来。

7.4 每个模型-题目组合目录里的 6 个核心 JSON

1. task_spec.json — 本次考试用的“考卷原件”

运行开始时，程序把当时生效的题目 JSON 原样复制一份存进结果目录。即使之后你修改了 tasks/*.json，旧结果里的这份快照仍然描述“当时到底考的是什么”。**复现历史结果时以这份为准。**

2. design_log.json — 模型的“草稿纸”

记录完整对话历史：系统提示词、每轮模型消息、每次工具调用的返回结果。字段结构：

```
task_id, task_spec_file, model, protocol_version
max_iterations / iterations_used / selected_iteration / stop_reason
model_calls / solve_calls
messages: [ {role, content}, ... ]    <- 完整对话
resource_summary / resource_usage_file
final_summary
```

适合回答这些问题：

- 模型每一轮提交了什么参数？
- 它为什么这样改？
- 哪一步失败了，错误信息是什么？

3. run_manifest.json — 后端的“过程台账”

结构化列出每个候选的完整生命周期：

```
candidates: [  
  {  
    iteration: 1,  
    status, success,  
    specification,          <- 模型原始提交  
    resolved_specification, <- 补默认值后的实际参数  
    design_parameters,      <- 带单位的输入  
    measured_metrics,       <- 带单位的实测  
    metrics,                <- 简化数值  
    project_file / metrics_file,  
    stages: { build, validation, solve, save, metrics } <- 每阶段成功与否  
  }, ...  
]
```

评测器只信任 run_manifest.json ，不读 design_log.json 。这保证评分只依据结构化结果，不受对话内容影响。

4. evaluation_report.json — 最终“成绩单”

最重要的文件，包含：

字段	含义
score / max_score	最终得分（默认满分 100）
passed	是否达到工程合格线（所有目标都过）
selected_iteration	被选中的最佳候选编号
selected_project_file	对应的 .aedt 工程路径
objective_evaluation	每个指标的实测值、是否通过、得分比例、得分
candidate_evaluations	所有可评分候选的逐项对比

字段	含义
best_score_trajectory	分数随迭代的走势
weakest_objective	最拖后腿的指标（改进优先看它）
engineering_passed / benchmark_score / geometric_mean_score	工程通过状态与基准分
checklist	逐项核对清单

5. resource_usage.json — 成本账本

分三块记录：model（API 延迟、token）、hfss（求解耗时、次数）、run（总耗时）。用于评估“这个模型贵不贵、慢不慢”。

6. manual_verification.json — 人工复核指引

列出选中工程的 AEDT 对象名、固定控制和逐项核验步骤。如果你想在 AEDT 图形界面里亲眼确认结果，按它做即可。

7.5 汇总层：两个 batch_results 文件

batch_results.json — 批次总索引

回答“这次批次跑了哪些题、都完成没有、报告在哪里”：

```
batch_id / status / execution_mode
started_at_utc / finished_at_utc
model_batch_file / model_batch_snapshot
max_iterations_per_model_per_task
tasks: [ { position, task_id, task_spec_file, task_report, status } ]
```

它不直接给分数，只给每道题的独立报告路径。

batch_results_<task_id>.json — 单题横向对比报告

这就是你现在打开的 batch_results_cp_patch_gps_1p575_v1.json 这类文件。它回答“这道题上每个模型考得怎么样”：

```

task_id, status, started_at_utc / finished_at_utc
max_iterations_per_model
models: [
  {
    position, model_id, config_file,
    result_directory,           <- 该模型该题的完整结果目录
    status,                     <- completed / skipped_disabled / ...
    model_name, api_style, temperature, reasoning_effort,
    stop_reason,                <- iteration_limit / finalize / error ...
    design_completed,
    evaluation_passed,          <- 是否工程合格
    score, max_score,           <- 得分
    selected_iteration,         <- 选中第几个候选
    selected_project_file,      <- 选中的 .aedt
    evaluation_report / run_manifest / resource_usage, <- 深挖路径
  }, ...
]
ranking: [ { rank, model_id, score, selected_iteration }, ... ]

```

阅读技巧：

1. 先看 `ranking`，一眼看出谁第一；
2. 再看 `models[].evaluation_passed` 和 `score`，知道是“接近合格”还是“差很远”；
3. 顺着 `selected_project_file / evaluation_report` 点进去，看最佳候选具体长什么样、哪一项拖分。

7.6 推荐阅读顺序（从粗到细）

想搞懂一次历史运行，按这个顺序看最省力：

- 第 1 步: batch_results.json (或 run_results.json)
→ 知道这次跑了什么、状态如何、去哪里看下一层
- 第 2 步: batch_results_<task_id>.json
→ 知道这道题各模型的排名、得分、选中候选
- 第 3 步: <模型>/<题目>/evaluation_report.json
→ 看总分、是否通过、每个指标的得分和 weakest_objective
- 第 4 步: <模型>/<题目>/run_manifest.json
→ 对比所有候选的参数和阶段状态，理解模型探索过程
- 第 5 步: candidate_NNN_metrics.json
→ 只看某个候选的详细实测指标
- 第 6 步: candidate_NNN.aedt
→ 在 AEDT 里打开几何，人工检查
- 第 7 步: design_log.json
→ 想理解“模型当时为什么这样改”时才看（文件很大）

如果只是想知道“这次考得怎么样”，看第 1–3 步就足够了。

7.7 其他历史目录

目录	说明
full_score_example/<题目>/candidate_XXX.aedt	每道题的满分见证工程， 用于证明题目有解
logs/design_log_*.json、 logs/evaluation_report_*.json	早期版本的零散日志，现在主流程已按 run/batch 目录归档
根目录 batch.log、 *.log	运行期控制台日志，用于排查崩溃， 不是结构化结果

8. 常用命令

1. 安装依赖（建议 Python 3.11）

```
python -m pip install -r requirements.txt
```

2. 验证本机 AEDT 能被 Python 连上（先做这个）

```
python verify_min.py
```

3. 单题 + 单模型（默认模型配置）

```
python main.py --task-spec tasks/inset_patch_2p45.json --max-iterations 10
```

4. 单题 + 多模型

```
python main.py --model-batch model_batch.json --task-spec tasks/inset_patch_2p45.json --max-iteratio
```

5. 多题 + 单模型

```
python main.py --task-batch task_batch.json --max-iterations 10
```

6. 多题 + 多模型（完整矩阵）

```
python main.py --model-batch model_batch.json --task-batch task_batch.json --max-iterations 10
```

7. 无模型、无 AEDT 的离线演示（快速验证流程）

```
python main.py --use-placeholder
```

8. 离线单元测试（不会启动 AEDT）

```
python -B -m unittest discover -s tests -v
```

`--max-iterations` 的含义是“最多完整处理多少个候选”，不是模型消息条数。每轮候选都包含建模、求解和读数。

9. 常见问题（FAQ）

Q1：为什么我使用最新版本的PyAEDT和Ansys Electronics Desktop（学生版），无法正常运行此项目？

Ansys Electronics Desktop的学生版存在一些官方承认的兼容问题，无法通过最新版的PyAEDT调用。目前已知Ansys Electronics Desktop Student 2025 R2 / HFSS 2025.2可以配合PyAEDT 0.19.0 稳定运行。

Q2：模型明明说“我成功了”，为什么 `evaluation_passed` 是 `false`？

模型的自述不算数。评测器只看 HFSS 实测值和结构化阈值。以 `evaluation_report.json` 为准。

Q3：为什么有的候选没有分数？

建模、校验、求解、保存、读数任一阶段失败的候选会被排除在评分之外。看 `run_manifest.json` 里该候选的 `stages` 就知道挂在哪一步。

Q4：分数很低，先看哪里？

先看 `evaluation_report.json` 的 `weakest_objective` 和 `objective_evaluation`，找得分比例最低的指标；再对比 `candidate_evaluations` 里各候选的趋势，判断是方向错了还是预算不够。

Q5：改了题目 JSON，旧结果会受影响吗？

不会。每次运行都会把当时的题目快照存进结果目录的 `task_spec.json`，旧成绩按旧考卷解释。

Q6：想比较两次批次的成绩，要注意什么？

确认题目合同、HFSS 版本和 `simulation_control` 完全一致；否则分数不可直接比较。`task_spec.json` 快照可以帮你逐字段比对。

Q7：AEDT 打不开或连不上怎么办？

先运行 `python verify_min.py`；检查 `.env` / 环境变量里的 `AEDT_ROOT` 是否指向本机安装目录（Student 2025 R2 默认示例见 `.env.example`）。

10. 修改代码前的建议路线

1. 只想换题/调难度 → 只改 `tasks/*.json`，先跑 `python main.py --use-placeholder` 看流程；
2. 想加一道新题（新拓扑）→ 同步准备：`frameworks/` 框架、可信 `builder`、任务 JSON、指标读取器、离线测试；
3. 想接入新模型 → 写一份 `job_xxx.yaml`，在 `model_batch.json` 注册，先用 `--max-iterations 1` 小规模验证；
4. 想改评分规则 → 改 `task_spec.py` / `evaluator.py`，务必跑 `tests/`；
5. 改了建模代码 → 先离线测试，再 `verify_min.py`，最后用低迭代真实任务验证。